



POLITECNICO
MILANO 1863



ITCNICO
LANO 1863

architettura dei calcolatori e sistemi operativi

**livello logico digitale – blocchi funzionali
combinatori**

RISC V – appendice a P&H

blocchi funzionali combinatori

esiste una ben nota e ormai stabilizzata **libreria di blocchi funzionali predefiniti** di tipo combinatorio che contiene i blocchi per tutte le funzioni combinatorie di base

- la libreria contiene anche blocchi funzionali di tipo sequenziale

i tipici blocchi funzionali combinatori sono

- ☐ multiplexer
- ☐ decoder (decodificatore)
- ☐ confrontatore
- ☐ semisommatore e sommatore (half adder e full adder)
- ☐ addizionatore a n bit
- ☐ ALU



multiplexer

il blocco funzionale multiplexer ha

- $n \geq 1$ ingressi di selezione
- $2^n \geq 2$ ingressi dati
- **1** uscita

gli *ingressi dati* sono numerati a partire da 0

$$k = 0, 1, 2, \dots, 2^n - 1$$

se sugli *ingressi di selezione* è presente la configurazione binaria corrispondente a k , il k^{esimo} ingresso dati viene presentato in uscita



multiplexer a un ingresso di selezione

un (n) ingresso di selezione, due (2^n) ingressi dati, un'uscita

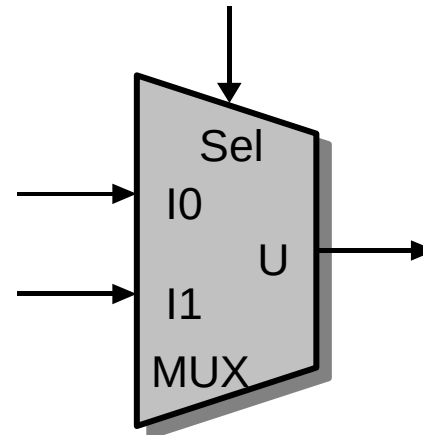
tabella della verità

I0	I1	Sel (Ctrl)	OUT
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

I0	I1	Sel (Ctrl)	OUT
D1	D2	0	D1
D1	D2	1	D2

In forma canonica come «somma di prodotti»

$$\text{OUT} = \text{Sel } !I0 \ I1 + !\text{Sel } I0 \ !I1 + !\text{Sel } I0 \ I1 + \text{Sel } I0 \ I1$$



...semplificando (con le regole dell'algebra di Boole)

$$\text{OUT} = \text{Sel } I1 + !\text{Sel } I0$$



multiplexer a due ingressi di selezione

Se $S_0 S_1 == 00$ in uscita il valore I_0

Se $S_0 S_1 == 01$ in uscita il valore I_1

etc.

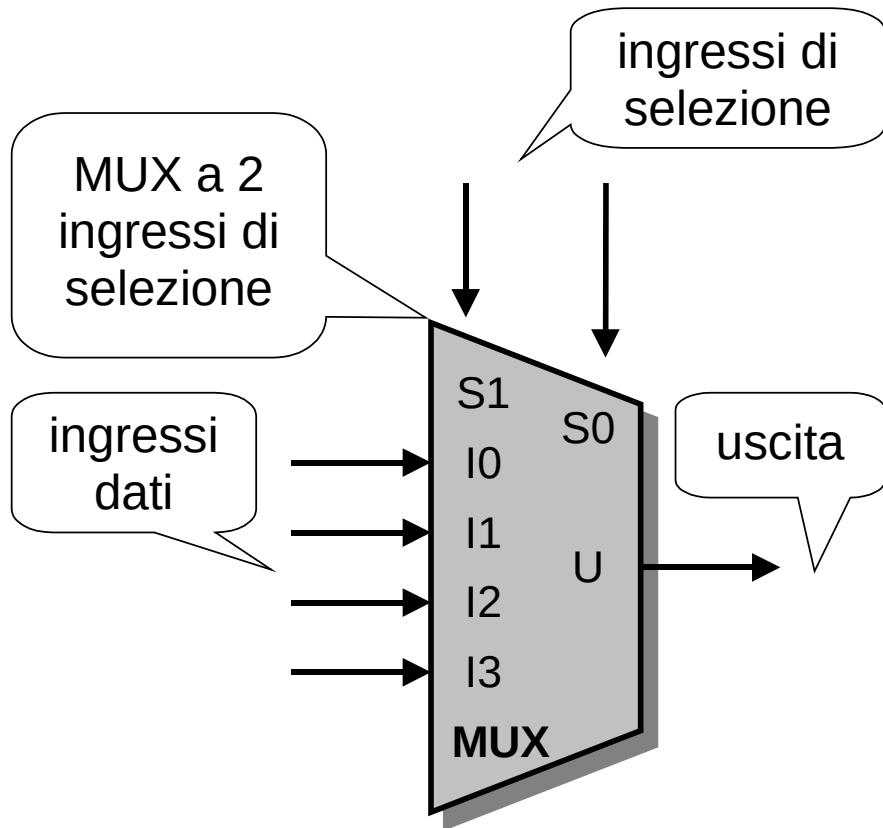


tabella delle verità

S_1	S_0	I_0	I_1	I_2	I_3	U
0	0	0	X	X	X	0
0	0	1	X	X	X	1
0	1	X	0	X	X	0
0	1	X	1	X	X	1
1	0	X	X	0	X	0
1	0	X	X	1	X	1
1	1	X	X	X	0	0
1	1	X	X	X	1	1

Funzione Booleana (non in forma canonica):

$$U = !S_1 \cdot !S_0 \cdot I_0 + !S_1 \cdot S_0 \cdot I_1 + S_1 \cdot !S_0 \cdot I_2 + S_1 \cdot S_0 \cdot I_3$$

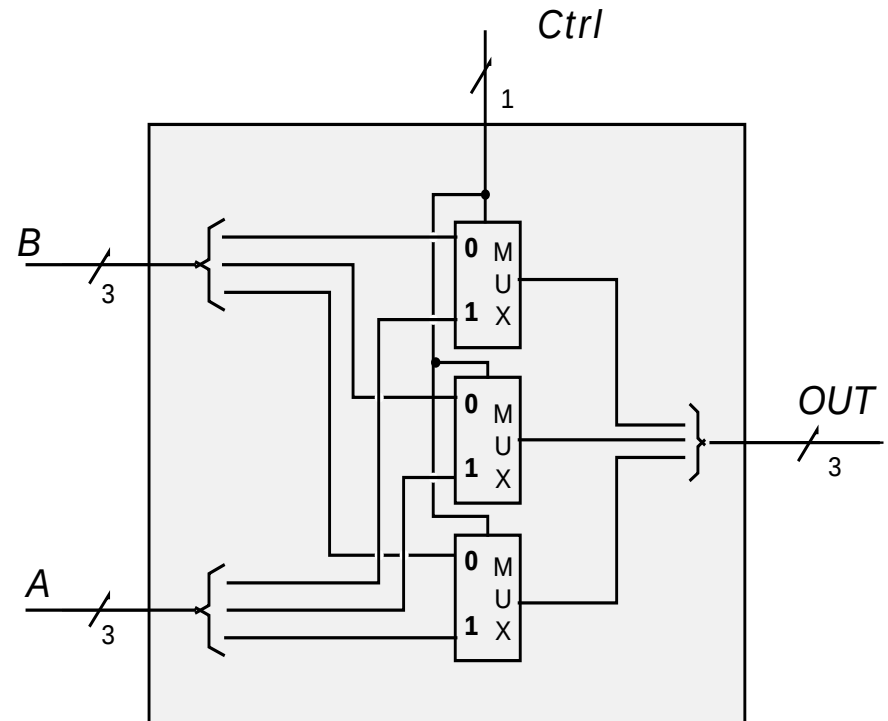
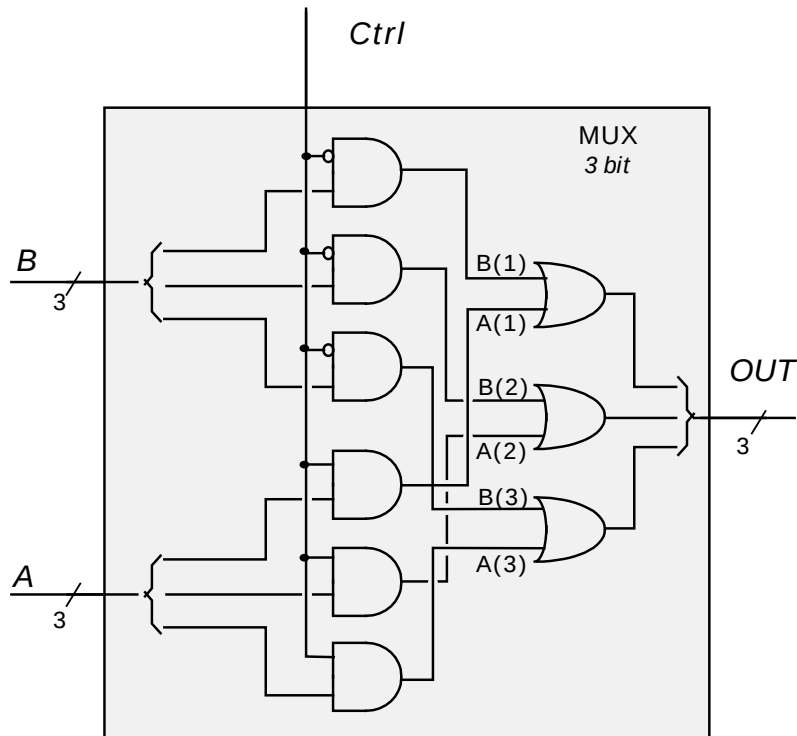


multiplexer a un ingresso di selezione e due ingressi dati da k bit

esempio: $k = 3$

B	A	Ctrl	OUT
b	a	0	b
b	a	1	a

$$OUT = !Ctrl \cdot B + Ctrl \cdot A$$



decodificatore (decoder) ... da codifica binaria (naturale) a codifica one-hot

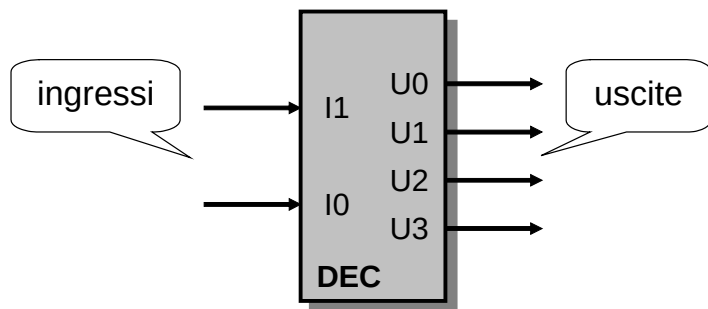
il blocco funzionale decodificatore ha

- $n \geq 1$ ingressi
- $2^n \geq 2$ uscite

le uscite sono numerate a partire da 0: $k = 0, 1, 2, \dots, 2^n - 1$

se sugli ingressi è presente il numero binario k , la k^{esima} uscita assume il valore 1 e le restanti uscite assumono il valore 0

decodificatore a due ingressi



I1	I0	U0	U1	U2	U3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

confrontatore (comparator)

il blocco funzionale confrontatore ha

- due gruppi A e B di ingressi da $n \geq 1$ bit ciascuno
- tre uscite: minoranza $A < B$, uguaglianza $A = B$ e maggioranza $A > B$

il blocco confronta i due numeri binari A e B da n bit presenti sui due gruppi di ingressi, e attiva (a 1) l'uscita corrispondente all'esito del confronto



confrontatore con ingressi a due bit

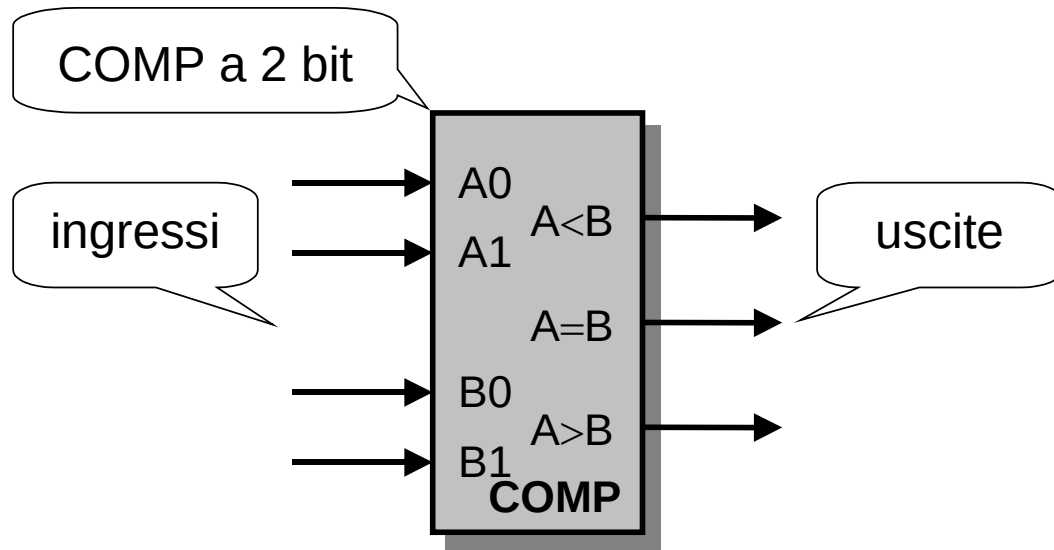


tabella delle verità

A1	A0	B1	B0	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0



blocchi aritmetici fondamentali

lavorano in binario **naturale** intero su $k \geq 1$ bit

- addizionatore a un bit
 - semisommatore – half adder
 - sommatore completo – full adder
- addizionatore a k bit in binario naturale intero



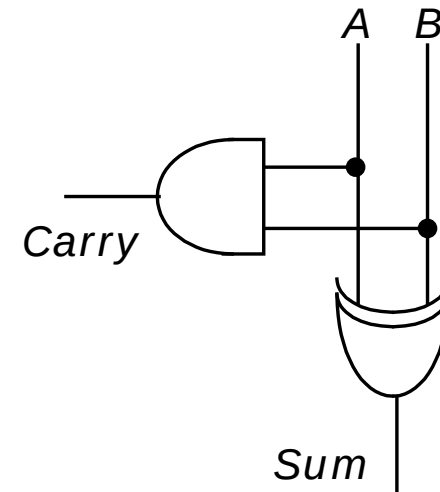
semisommatore con addendi a 1 bit – half adder

HALF-ADDER

<i>A</i>	<i>B</i>	<i>Carry</i>	<i>Sum</i>
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$Sum = !A \cdot B + A \cdot !B = A \oplus B$$

$$Carry = A \cdot B$$



sommatore completo – full adder

FULL ADDER

A	B	Carry In (C)	Sum S	Carry Out (C_out)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S = A \text{ xor } B \text{ xor } C$$

$$C_out = A B + C (A \text{ xor } B)$$

$$\begin{aligned} S &= !A !B C + !A B !C + A !B !C + A B C \\ &= C (!A !B + A B) + !C (!A B + A !B) \\ &= C !(A \text{ xor } B) + !C (A \text{ xor } B) \end{aligned}$$

ponendo $Z = A \text{ xor } B$

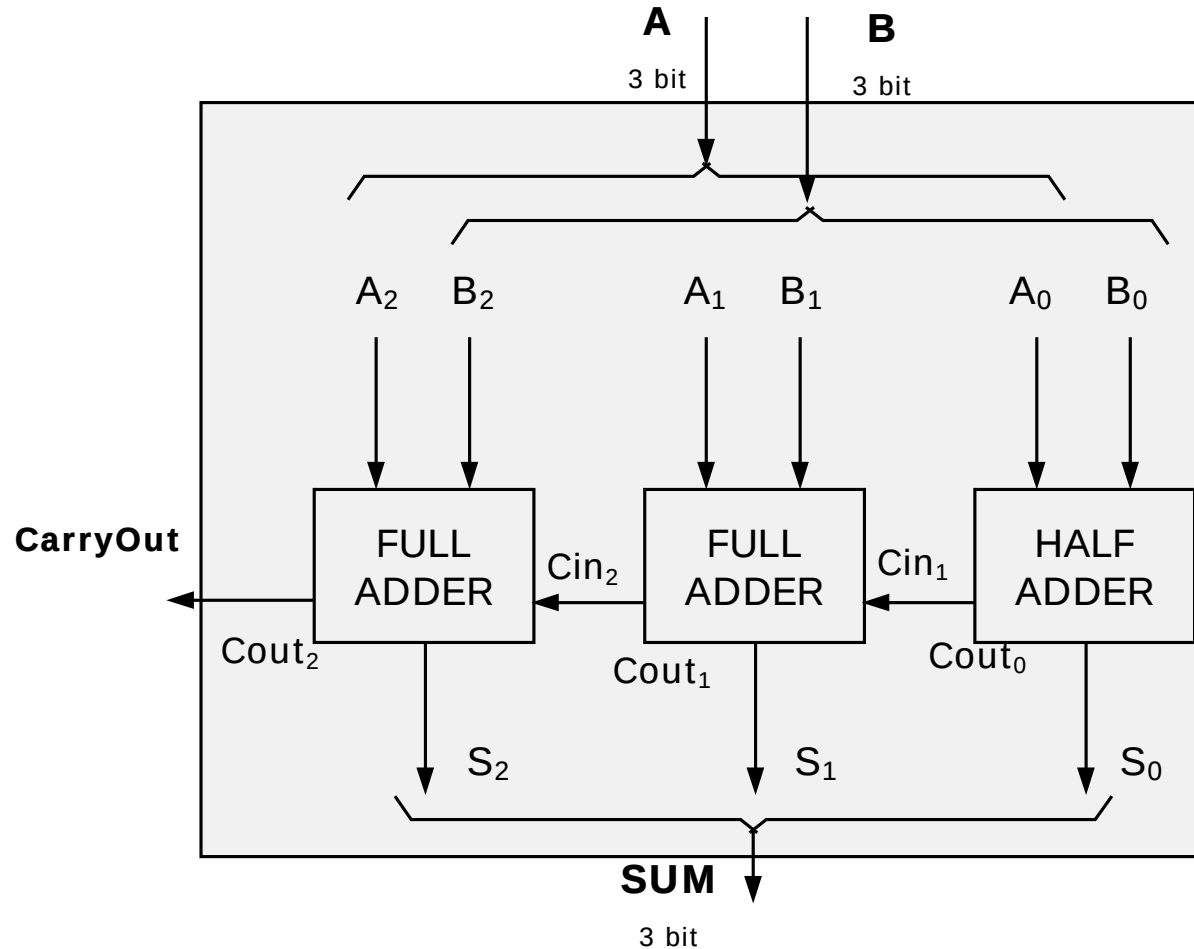
$$\begin{aligned} &= !Z C + Z !C = Z \text{ xor } C \\ &= A \text{ xor } B \text{ xor } C \end{aligned}$$

$$\begin{aligned} C_out &= !A B C + A !B C + A B !C + A B C \\ &= C (!A B + A !B) + A B (!C + C) \\ &= C (A \text{ xor } B) + A B \end{aligned}$$

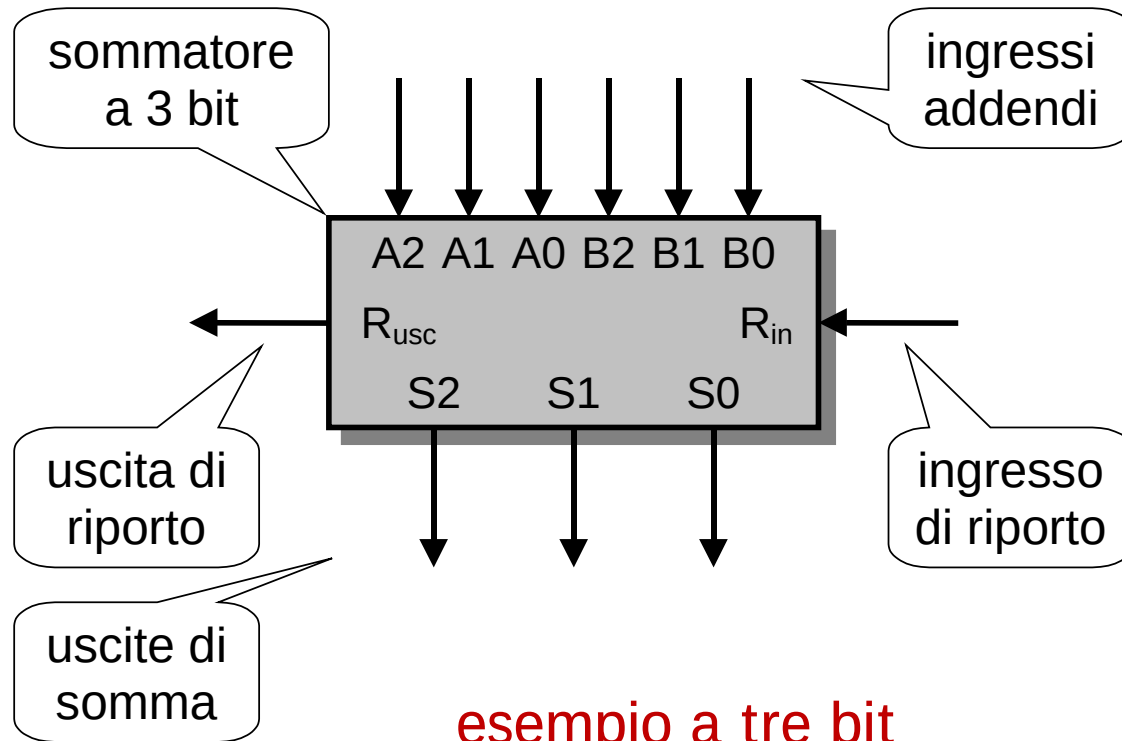
..... schema rete da ricavare



addizionatore a k bit in binario naturale intero



sommatore binario naturale intero a n bit



somma (A+B) e sottrazione (A-B) in CPL2

Operandi:

- A e B in CPL2 su k bit

Risultato di somma:

- un valore R in CPL2 su k bit (ottenuto con un sommatore per binario naturale)
- il valore del bit di overflow OVFL (=1 se operandi hanno lo stesso segno (MSbit) e questo è discorde con quello di R; =0 altrimenti)

Per la sottrazione si procede

- calcolando $-B$ in CPL2 (come il risultato di $\text{not}(B)+1$ su k bit)
- calcolando $A+(-B)$

sommatore/sottrattore in CPL2:

- sommatore binario naturale intero a k bit con ingresso **carry_in**
- segnale **!add/sub** (cioè varrà 0 in caso di add, 1 in caso di sub) in ingresso a ciascuna porta di una batteria di **xor** su ingresso B per effettuare (se necessario) il calcolo dell'opposto con anche il $\text{carry_in} == 1$ e ottenere $A+(-B)$
- rete combinatoria per generare segnale di OVFL



ALU e RISC V

istruzioni aritmetiche

- *add*
- *sub*

istruzioni logiche

- *or*
- *and*
- *nor*

istruzioni *slt* e *sltu*



ALU RISC V a un bit

AND, OR, somma e sottrazione in complemento a 2

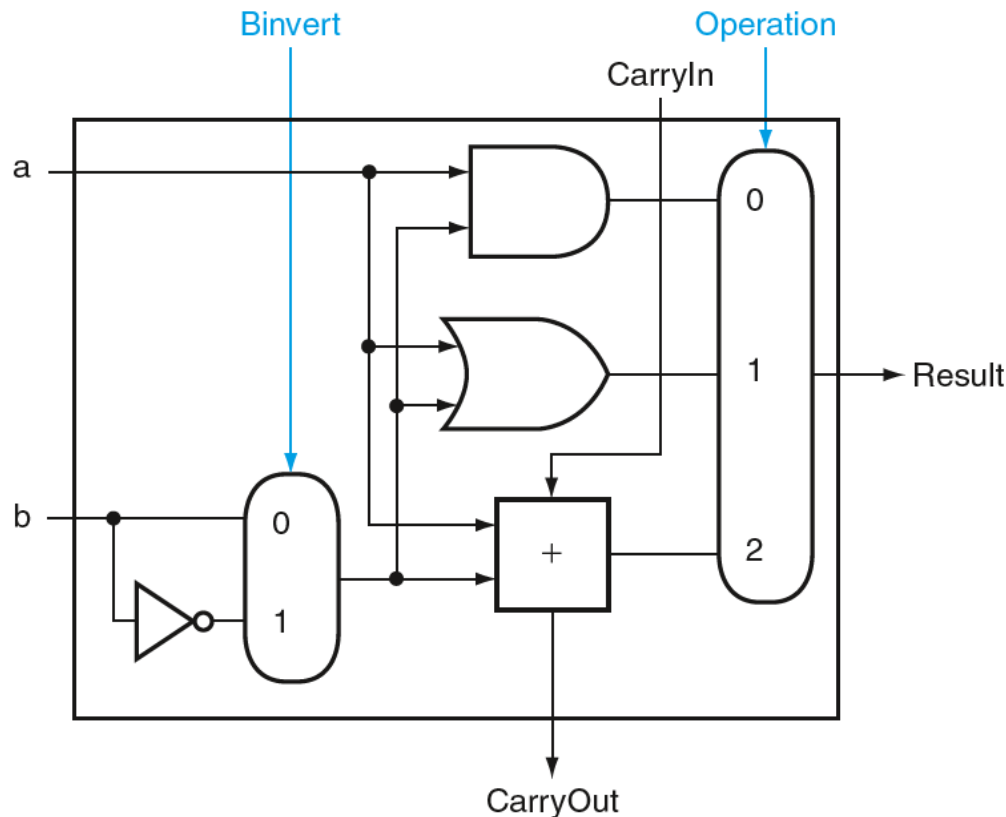


FIGURE A.5.8 A 1-bit ALU that performs AND, OR, and addition on *a* and *b* or *a* and $\bar{b}$. By selecting $\bar{b}$ (*Binvert* = 1) and setting *CarryIn* to 1 in the least significant bit of the ALU, we get two's complement subtraction of *b* from *a* instead of addition of *b* to *a*.

ALU RISC V a 64 bit

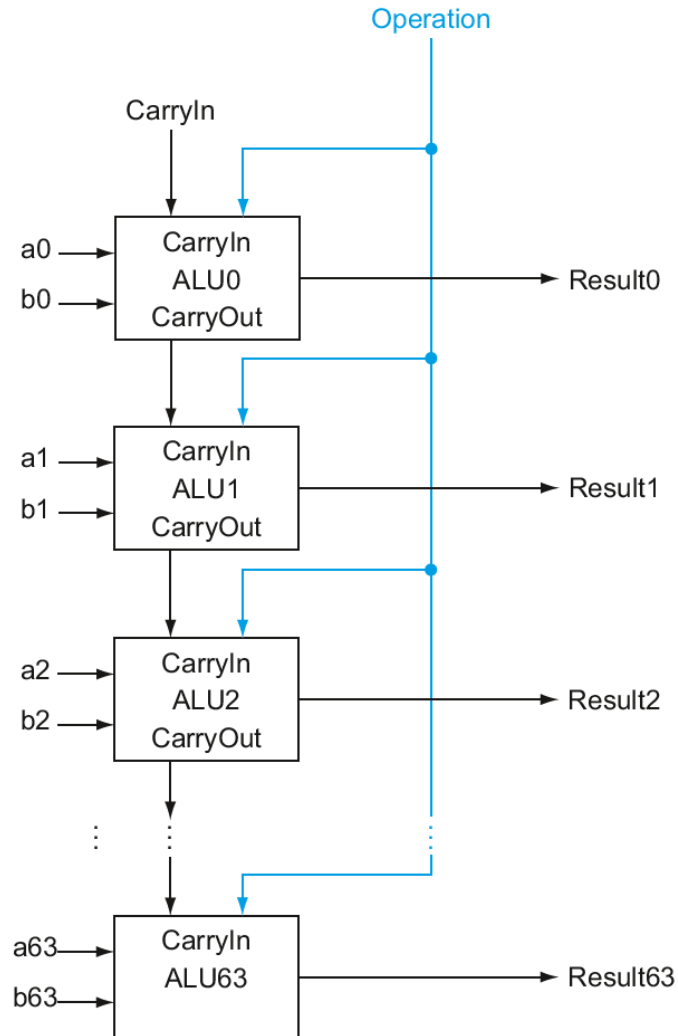


FIGURE A.5.7 A 64-bit ALU constructed from 64 1-bit ALUs. CarryOut of the less significant bit is connected to the CarryIn of the more significant bit. This organization is called ripple carry.



si aggiunge il NOR

$$\overline{A + B} = \overline{A} \overline{B}$$

schema ALU a un bit valido ad eccezione del bit più significativo, a cui va aggiunta la gestione del segnale di **overflow**

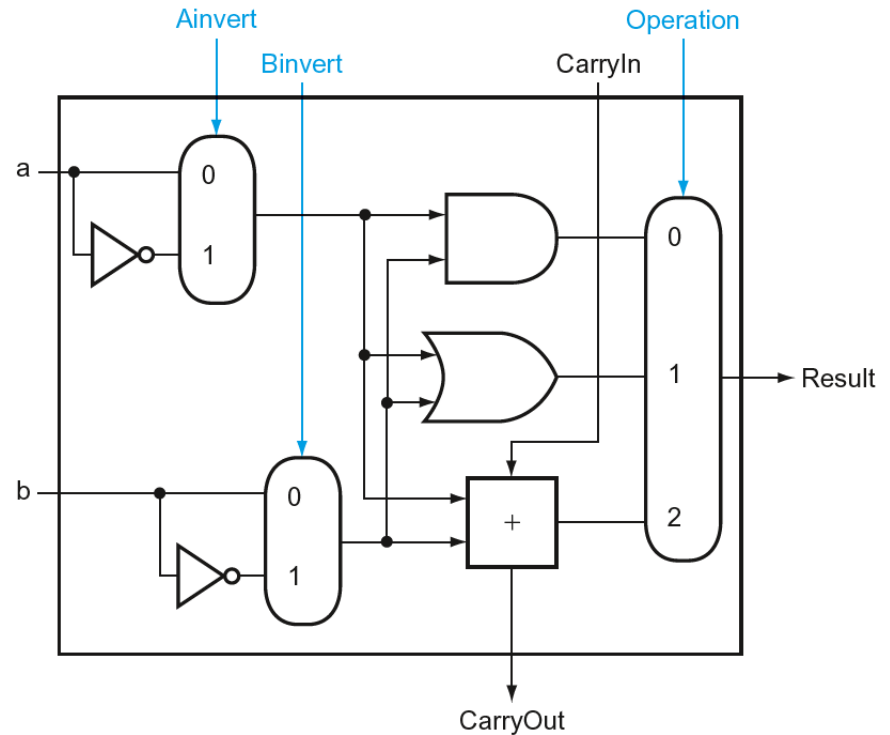


FIGURE A.5.9 A 1-bit ALU that performs AND, OR, and addition on a and b or $\bar{a}$ and $\bar{b}$. By selecting $\bar{a}$ ($A_{\text{invert}} = 1$) and $\bar{b}$ ($B_{\text{invert}} = 1$), we get a NOR b instead of a AND b .



rilevare il traboccamento (overflow detection): nel circuito di calcolo del bit più significativo

schema ALU a 1 bit in CPL2 per MSbit

il segnale di **Overflow** va a 1 se

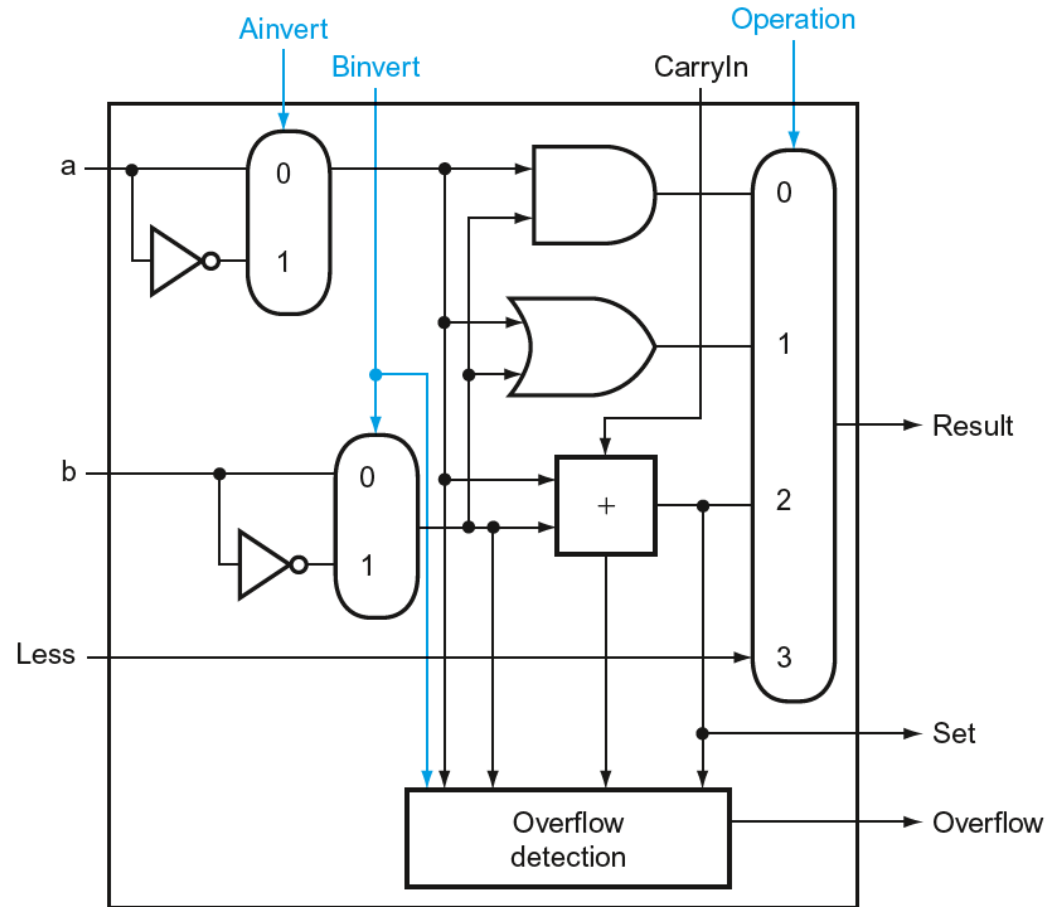
Msbith oper_1	Msbith oper_2	Msbith risultato
0	0	1
1	1	0

dove

oper_1 = **a** per somma e sottrazione

oper_2 = **b** per somma e

oper_2 = **b_negato** per sottrazione



la realizzazione di *slt*

Nota: questo è un valore a 64bit

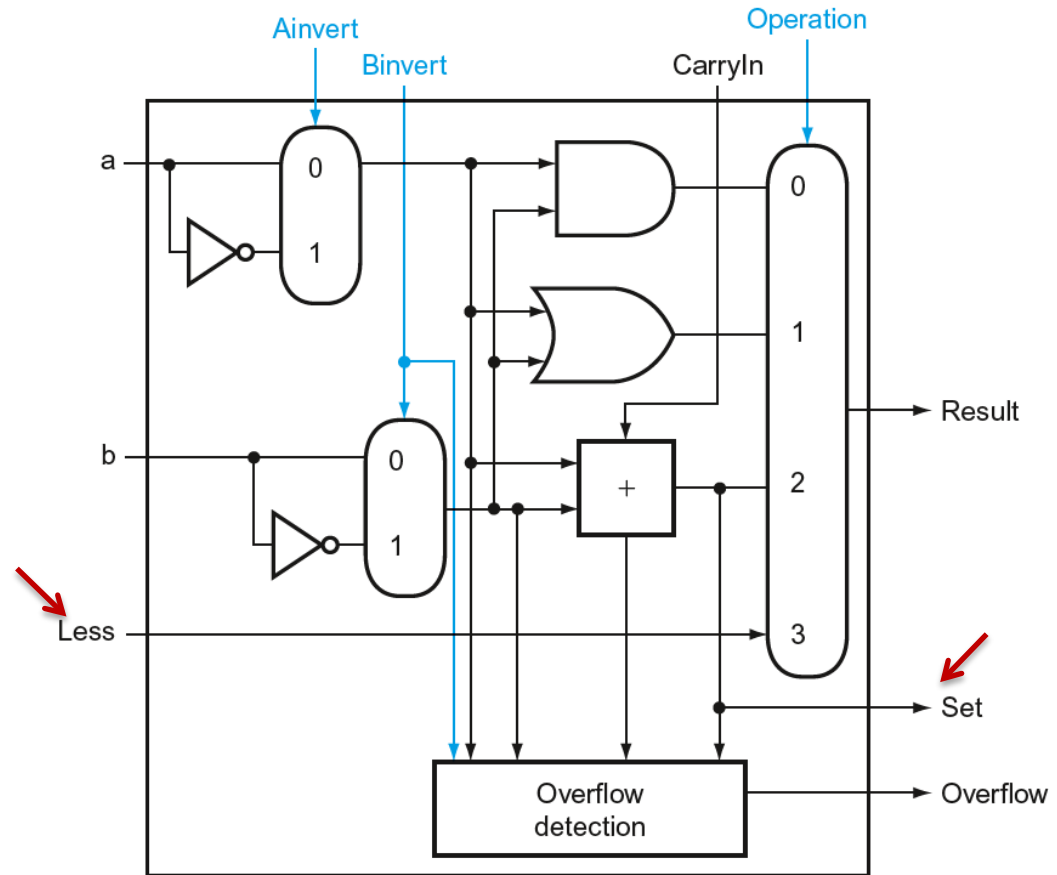
***slt* x1, x2, x3**

se $x2 < x3$ allora $x1 = 1$
altrimenti $x1 = 0$

$x2 < x3$ equivale a **$x2 - x3 < 0$**

se $x2 - x3 < 0$ allora il bit di segno del sommatore è 1 altrimenti è 0

l'ingresso ***less*** viene portato direttamente in uscita tramite il multiplexer e deve essere **0** per i **63** bit più significativi di $x1$ e **1** oppure **0** per il bit meno significativo (***Set*** sarà questo valore – calcolato nel circuito del MSb e riportato come valore del LSb).



.... la sottrazione (**$x2 - x3$**) può generare OVFL

$A < 0, -B < 0, R > 0$ (OVFL = 1) allora **Set = 1** perché $A < B$ e *slt* deve restituire/restituisce 1

$A > 0, -B > 0, R < 0$ (OVFL = 1) allora **Set = 0** perché $A > B$ e *slt* restituisce 0

..... se la sottrazione NON genera overflow (cioè OVFL = 0) allora **Set = Msbit di R**



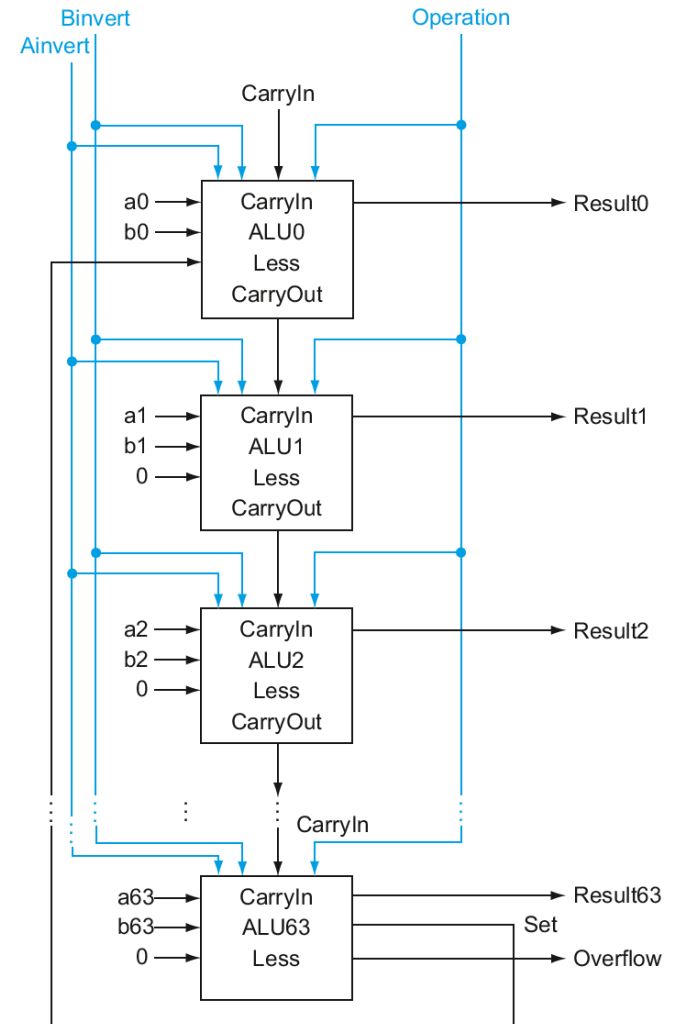
... a 64 bit

Overflow

È indicato da un segnale che esce solo dal bit più significativo

slt

ecco i valori di tutti i bit



... ulteriori raffinamenti

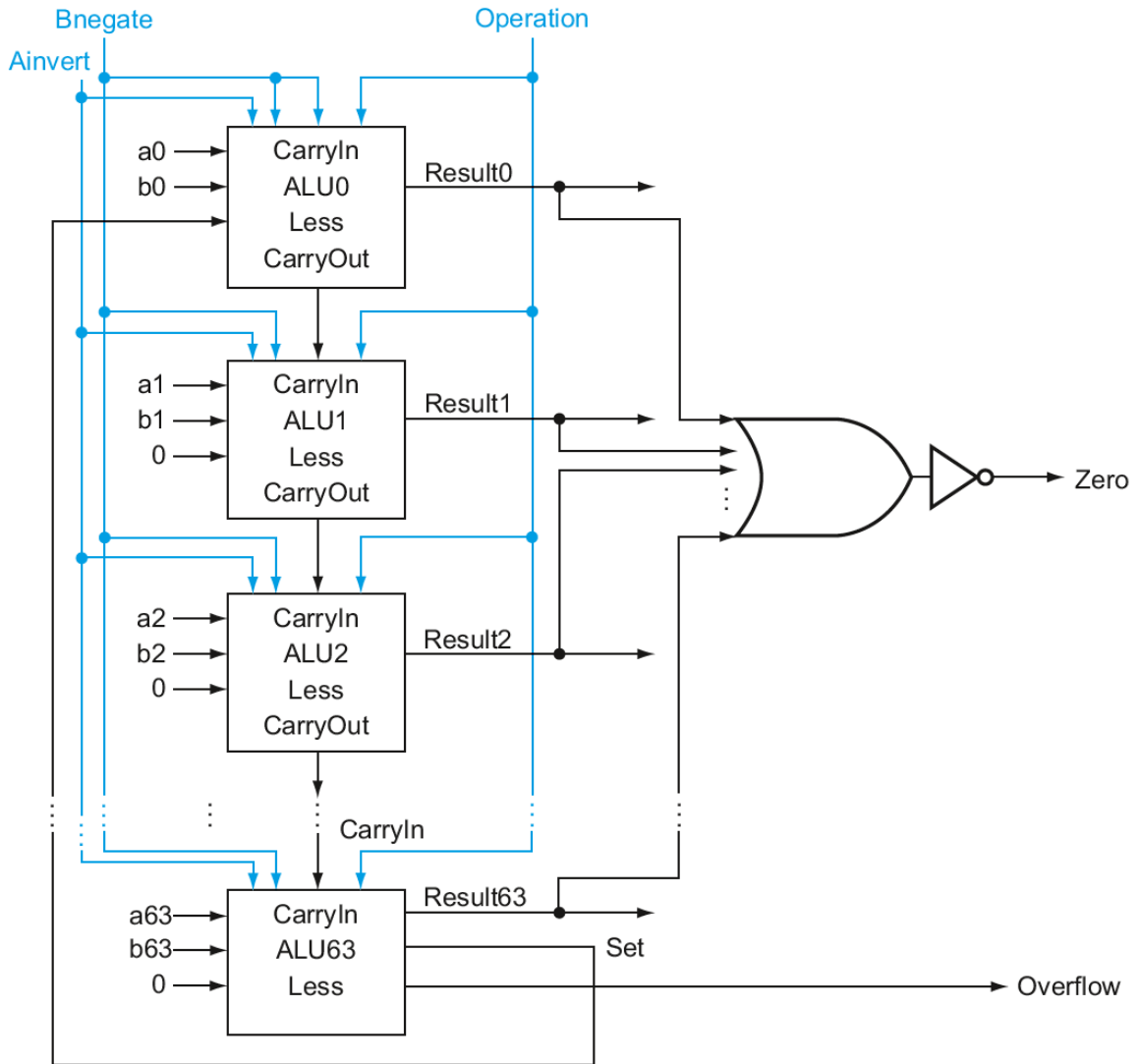
bit **Zero**

- posto a 1 se il risultato dell'ALU vale zero
- altrimenti a 0

Bnegate

per la sottrazione in CPL2 è necessario avere i segnali **Binvert** e **Carryin** a 1, mentre per le altre operazioni considerate (*eccetto il NOR...*), sono entrambi a 0

vengono sostituiti dall'unico segnale **Bnegate**

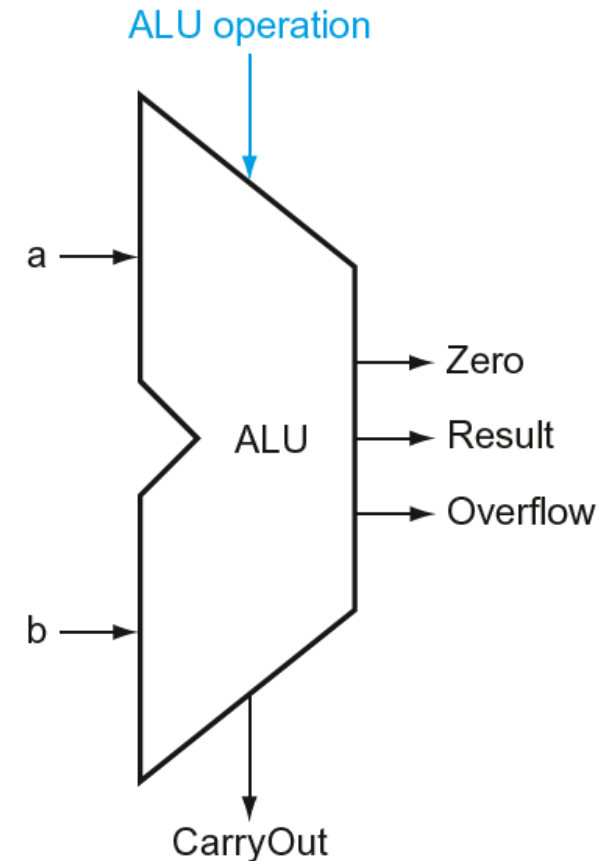


ALU di RISC V e linee di controllo

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR

le linee di controllo interne della ALU sono:

Ainvert, Bnegate e Operation (2 bit)



... come blocco funzionale